Window functions and streaming queries

DATA TRANSFORMATION WITH SPARK SQL IN DATABRICKS



Disha Mukherjee
Lead Data Engineer

datacamp

When groupBy isn't enough



- `groupBy()` → one row per group (totals, averages)
- **Window function** → adds a column, keeps every row
- **Use case:** running totals, rankings, row comparisons

Running total

```
from pyspark.sql.window import Window

window_spec = (
    Window.partitionBy("Customer_ID")
    .orderBy("Date")
    .rowsBetween(Window.unboundedPreceding, Window.currentRow)
)

df_running = df_valid.withColumn(
    "running_total",
    F.round(F.sum("Transaction_Amount").over(window_spec), 2)
)
```

Running total

```
+-----------+-------------------+------------------+-------------+
|Customer_ID|Date               |Transaction_Amount|running_total|
+-----------+-------------------+------------------+-------------+
|CUST003    |2023-01-03 00:00:00|5752.36           |5752.36      |
|CUST003    |2023-02-14 00:00:00|12340.00          |18092.36     |
+-----------+-------------------+------------------+-------------+
```

Ranking customers

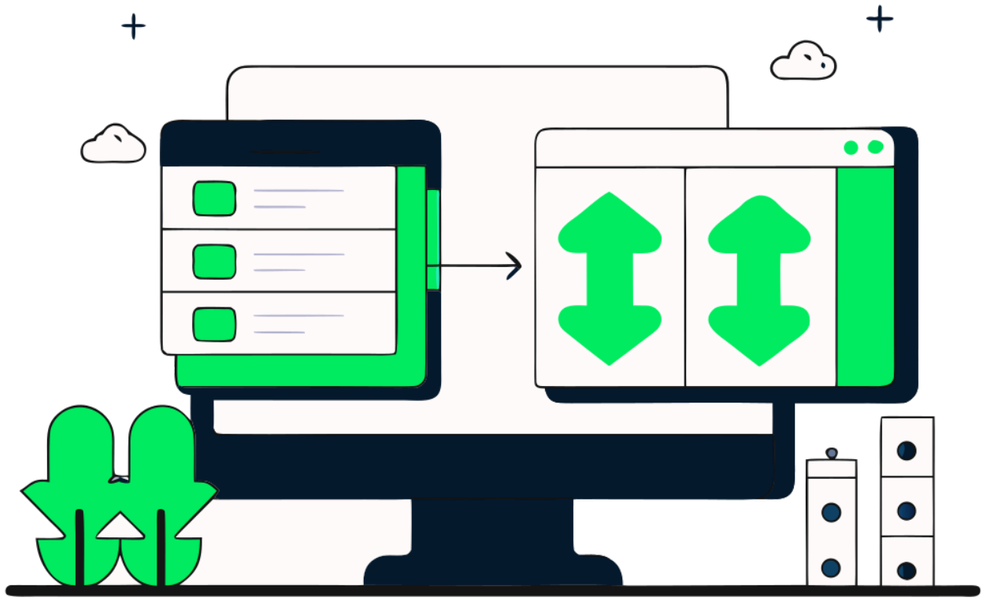
```
customer_totals = (
    df_valid.groupBy("Customer_ID")
    .agg(F.round(F.sum("Transaction_Amount"), 2).alias("total_revenue"))
)

rank_window = Window.orderBy(F.col("total_revenue").desc())
df_ranked = customer_totals.withColumn("revenue_rank", F.rank().over(rank_window))
```

```
+-----------+-------------+------------+
|Customer_ID|total_revenue|revenue_rank|
+-----------+-------------+------------+
|CUST1469   |99996.20     |1           |
|CUST19129  |99990.14     |2           |
|CUST39417  |99982.21     |3           |
+-----------+-------------+------------+
```

What is streaming?

- **Batch** - process a fixed dataset all at once
- **Streaming** - process data incrementally as it arrives
- Ideal for transaction feeds, logs, and event data



File-based streaming

```
Stream directory:
  day_1.csv  (108 KB)
  day_2.csv  (108 KB)
  day_3.csv  (108 KB)
  day_4.csv  (108 KB)
  day_5.csv  (108 KB)
```

- Directory of CSV files = streaming source
- Each new file = one micro-batch
- New files picked up automatically
- Schema must be defined explicitly

Reading the stream

```
df_stream = (
    spark.readStream.format("csv")
    .option("header", "true")
    .schema(streaming_schema)
    .load(STREAM_DIR)
)

print(df_stream.isStreaming)
```

```
True
```

What are checkpoints?



- Checkpoint = metadata directory on disk
- Records which files have been processed
- On restart, Spark resumes where it left off
- Prevents duplicate processing

Writing the stream

```
query = (
    df_stream.writeStream.format("delta")
    .outputMode("append")
    .option("checkpointLocation", CHECKPOINT_DIR)
    .option("path", DELTA_PATH)
    .trigger(availableNow=True)
    .start()
)
query.awaitTermination()
```

```
Status:       Stopped
Rows written: 5,000
```

Monitoring: status and lastProgress

```
print(query.status)


progress = query.lastProgress
print(f"Rows processed: {progress['numInputRows']}")
print(f"Rows/sec: {progress['processedRowsPerSecond']:.0f}")
```

```
{'message': 'Stopped', 'isDataAvailable': False, 'isTriggerActive': False}
Rows processed: 5,000
Rows/sec:       752
```

Checkpoint recovery

```
query_restart = (
    df_stream.writeStream.format("delta")
    .option("checkpointLocation", CHECKPOINT_DIR)
    .option("path", DELTA_PATH)
    .trigger(availableNow=True)
    .start()
)
query_restart.awaitTermination()
```

```
Rows on restart: 0
```

Let's practice!

DATA TRANSFORMATION WITH SPARK SQL IN DATABRICKS

Production pipelines with workflows

DATA TRANSFORMATION WITH SPARK SQL IN DATABRICKS



Disha Mukherjee
Lead Data Engineer

datacamp

Why Delta Lake?



- **ACID transactions** → roll back failed writes
- **Schema enforcement** → block mismatched types
- **Versioning** → query any previous state

Writing to Delta

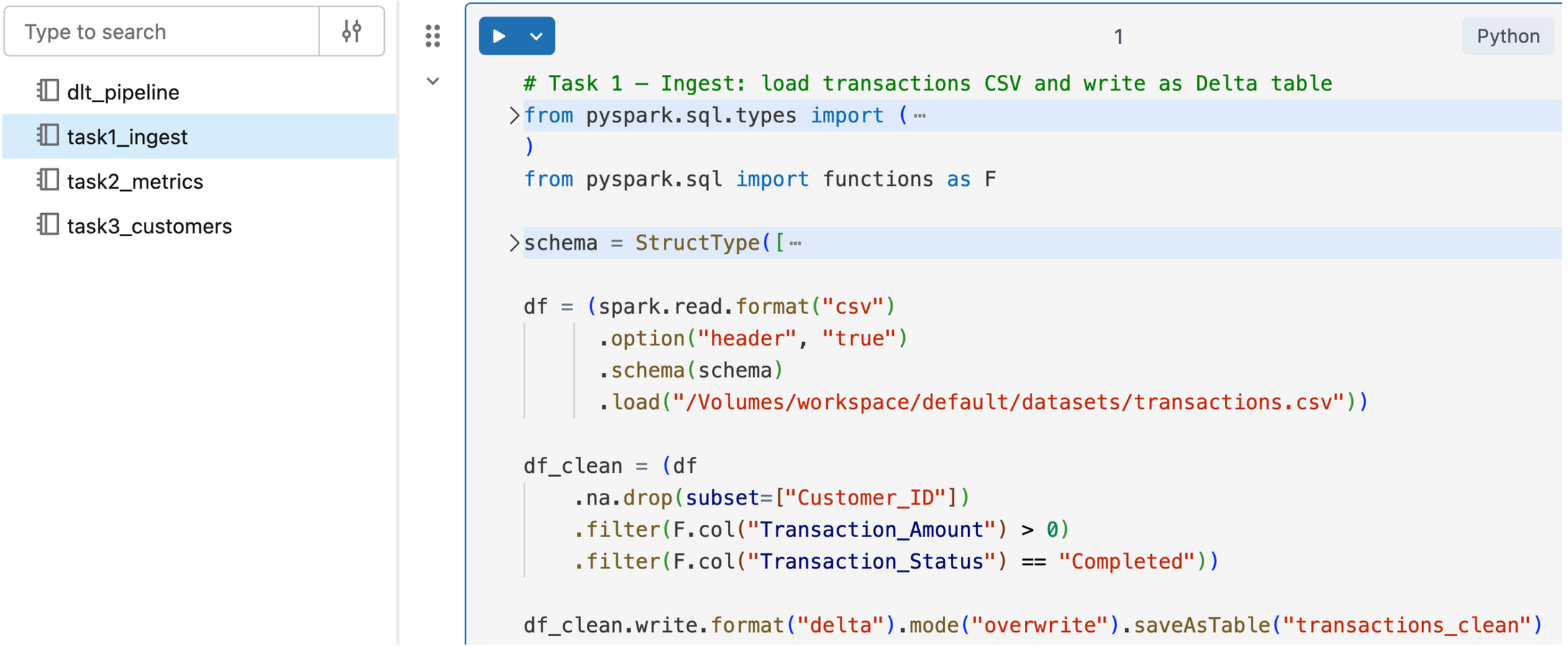
```
df_valid.write.format("delta") \
    .mode("overwrite") \
    .saveAsTable("transactions_clean")

print(f"Rows written: {df_valid.count():,}")
```

```
Rows written: 33,223
```

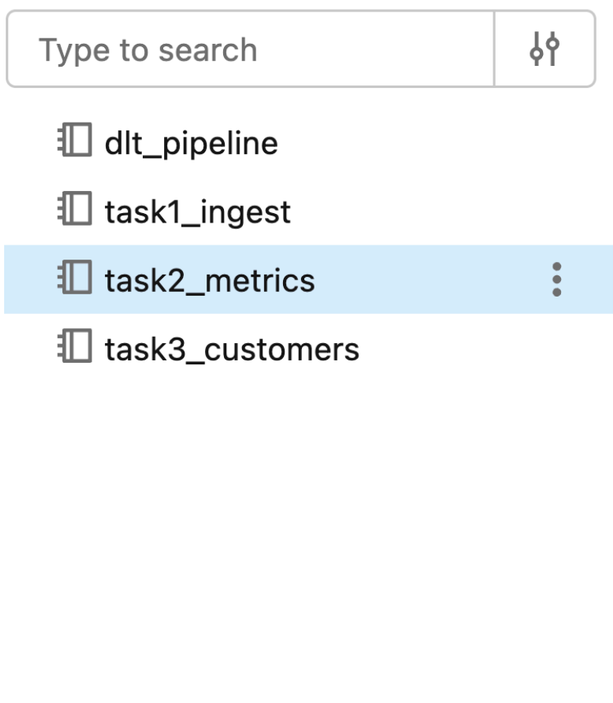
- A new Delta table appears in Unity Catalog

Notebook tasks



- Loads CSV → applies cleaning → writes a Delta table

Notebook tasks



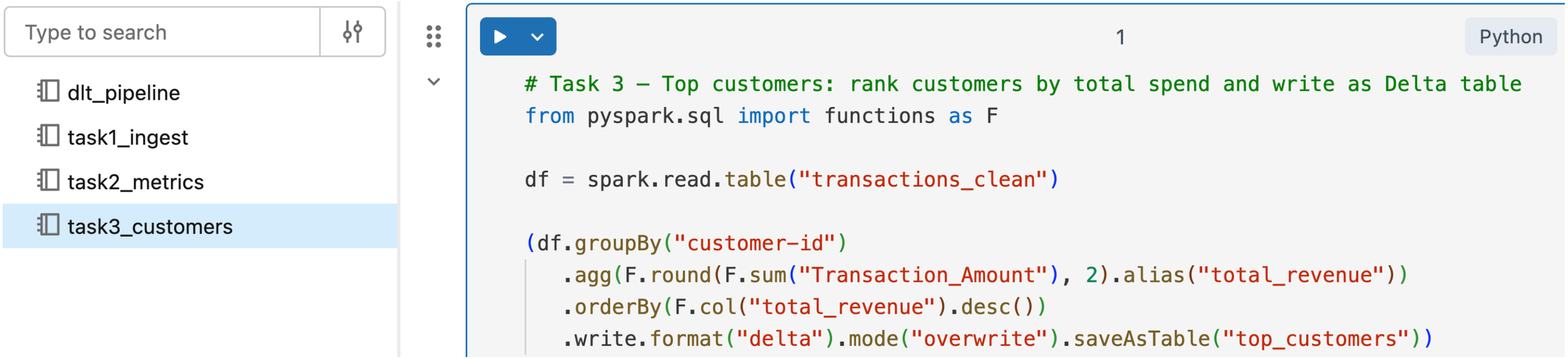

```python
# Task 2 — Metrics: compute revenue by category and write as Delta table
from pyspark.sql import functions as F

df = spark.read.table("transactions_clean")

(df.groupBy("Category")
   .agg(
      F.round(F.sum("Transaction_Amount"), 2).alias("total_revenue"),
      F.count("*").alias("transaction_count")
   )
   .orderBy(F.col("total_revenue").desc())
   .write.format("delta").mode("overwrite").saveAsTable("category_metrics"))
```

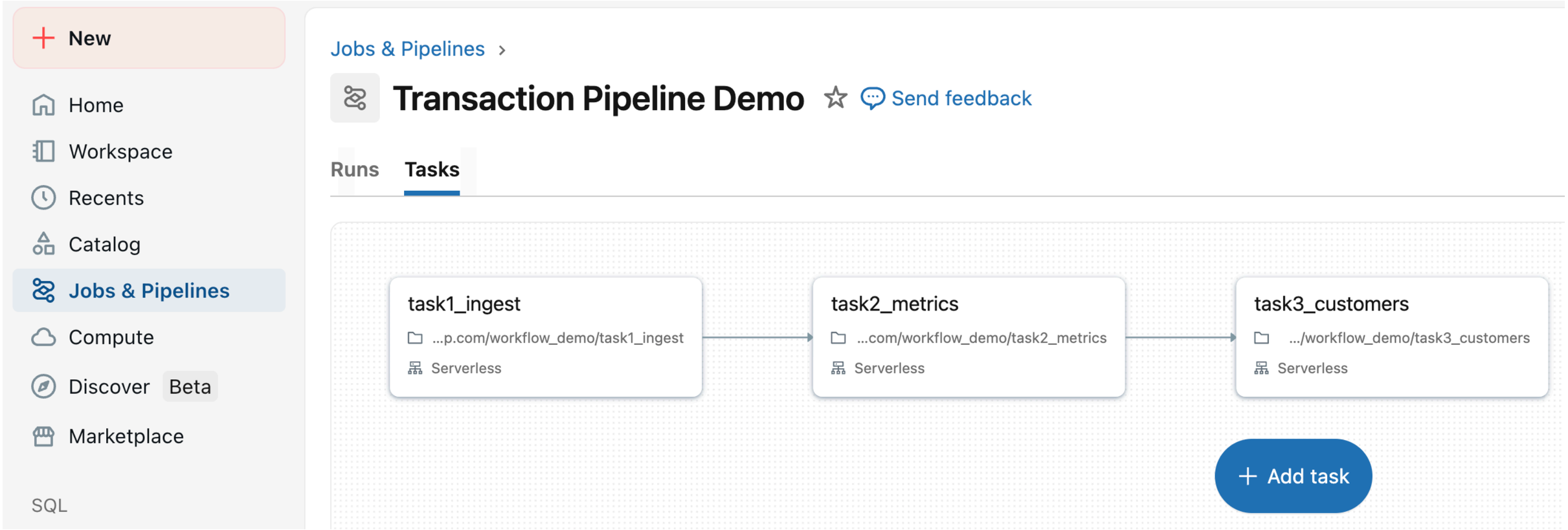
- Reads cleaned table → computes metrics → writes to a new table

Notebook tasks



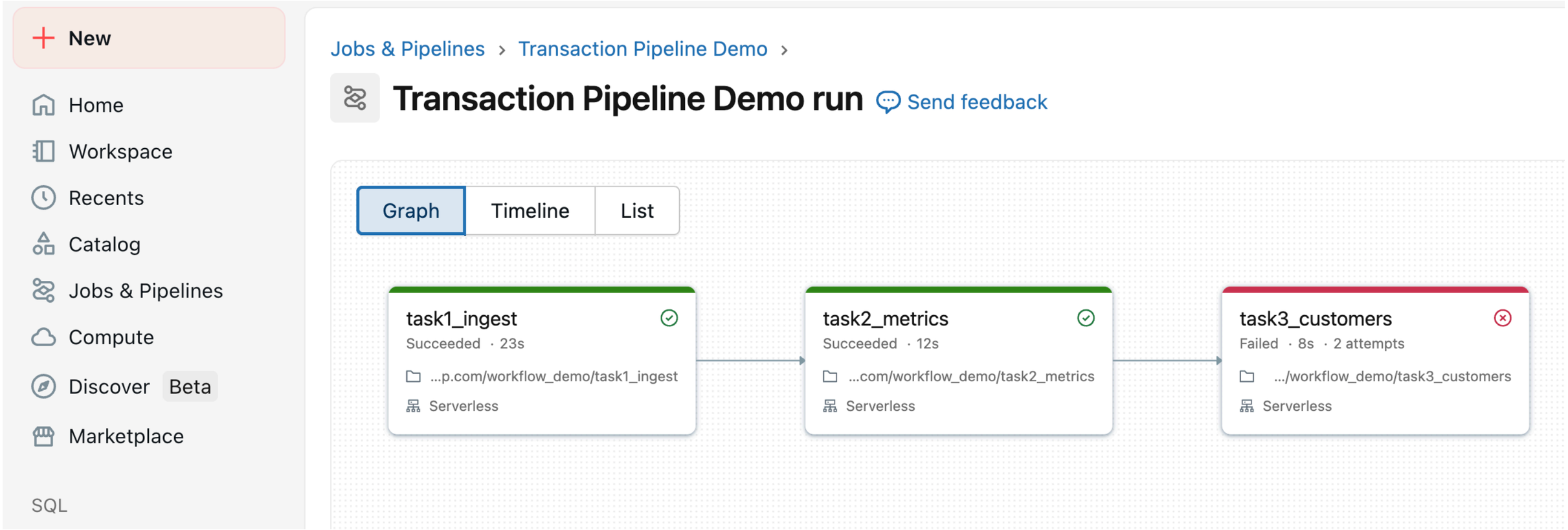
- Reads cleaned table → ranks customers → saves to a new table

Creating the job

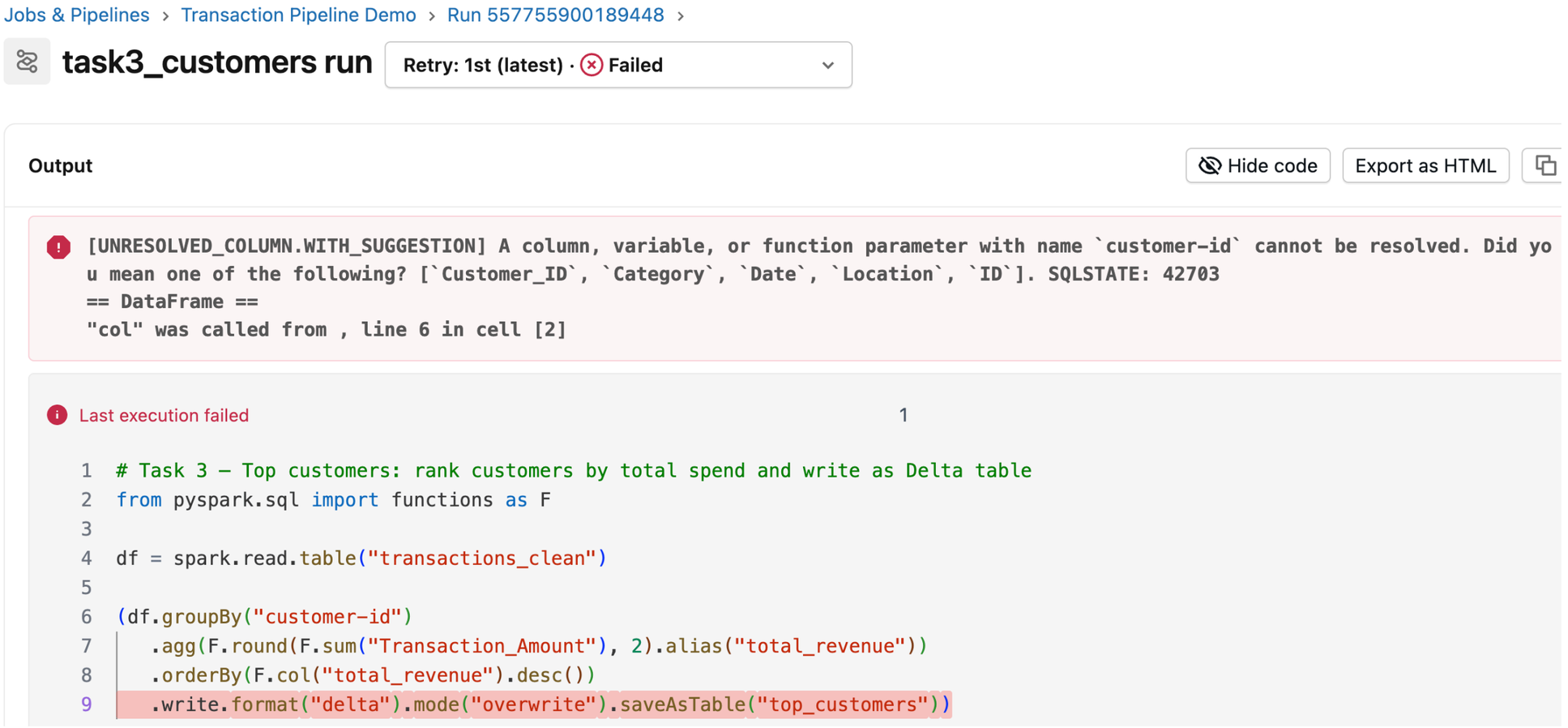


- Can run manually, schedule jobs, and set triggers

Running the job

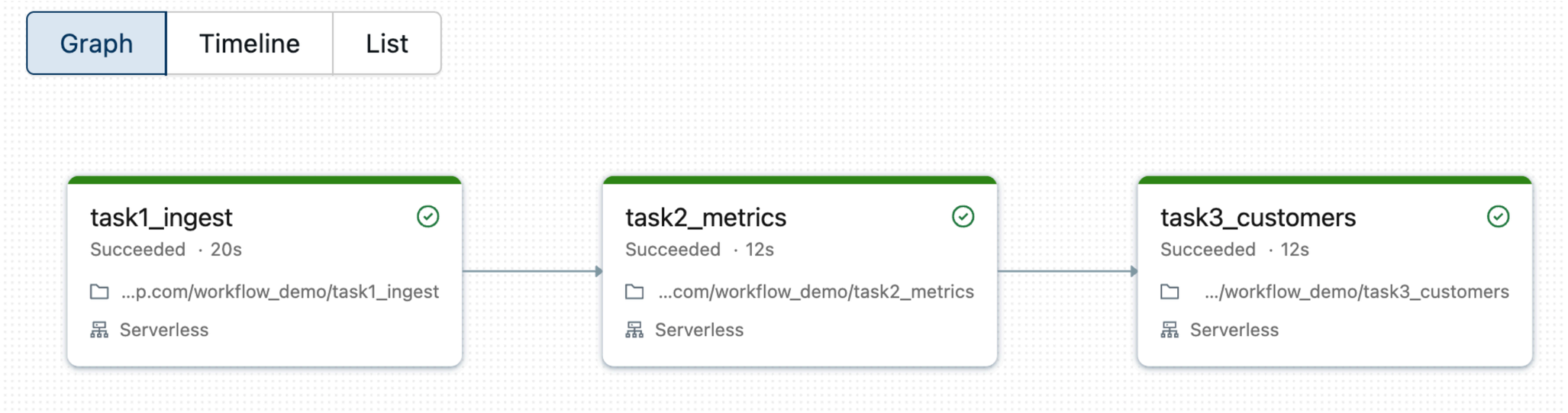


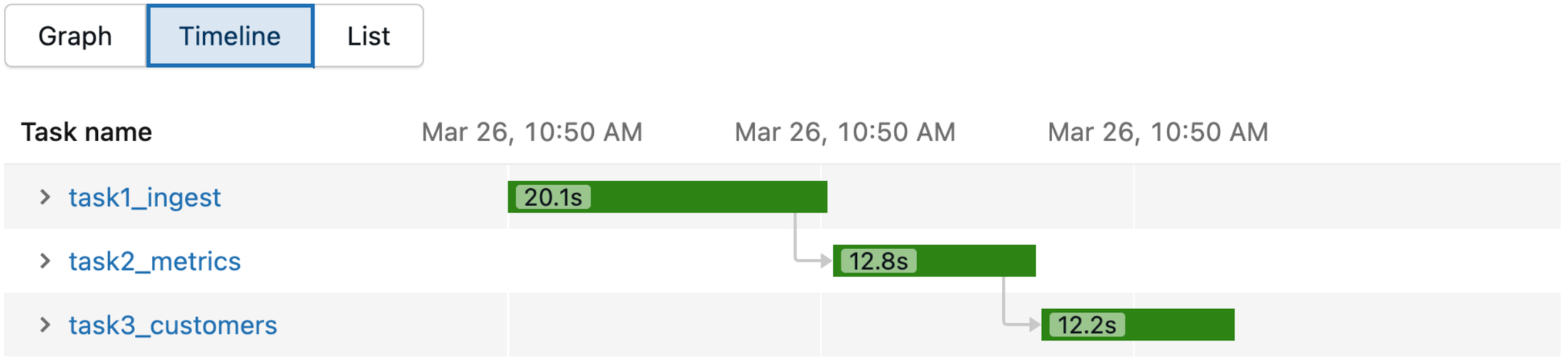
Running the job



- Error: `customer-id` should be `Customer_ID`

Running the job





What is Lakeflow?



- **Jobs** → we manage each step
- **Lakeflow** → declare what tables should contain
- Databricks handles order, retries, and compute

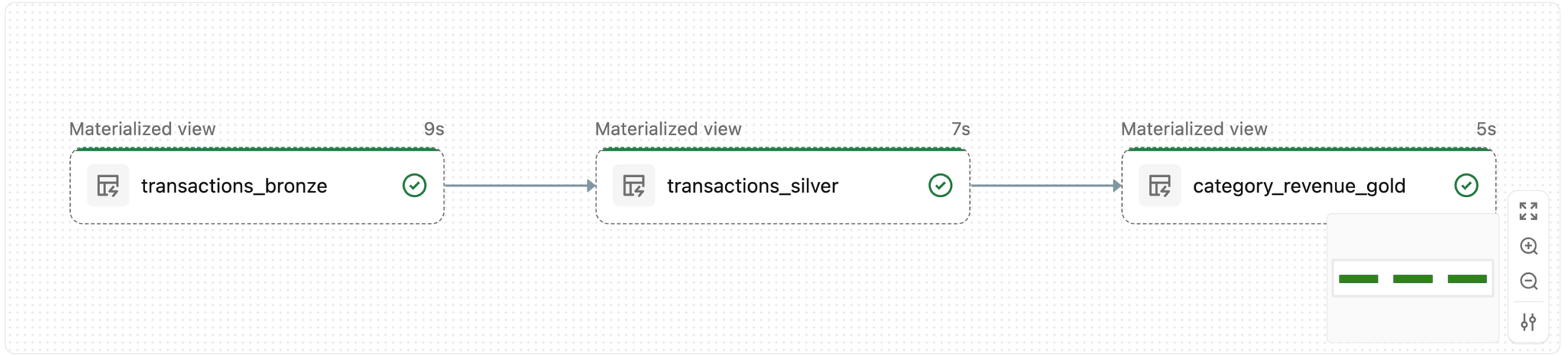
The @dlt.table pattern

```
@dlt.table(name="transactions_bronze")
def transactions_bronze():
    return spark.read.format("csv").schema(schema).load(FILE_PATH)

@dlt.table(name="transactions_silver")
def transactions_silver():
    return dlt.read("transactions_bronze").na.drop(...).filter(...)

@dlt.table(name="category_revenue_gold")
def category_revenue_gold():
    return dlt.read("transactions_silver").groupBy("Category").agg(...)
```

Pipeline run



Tables 3 Performance 3

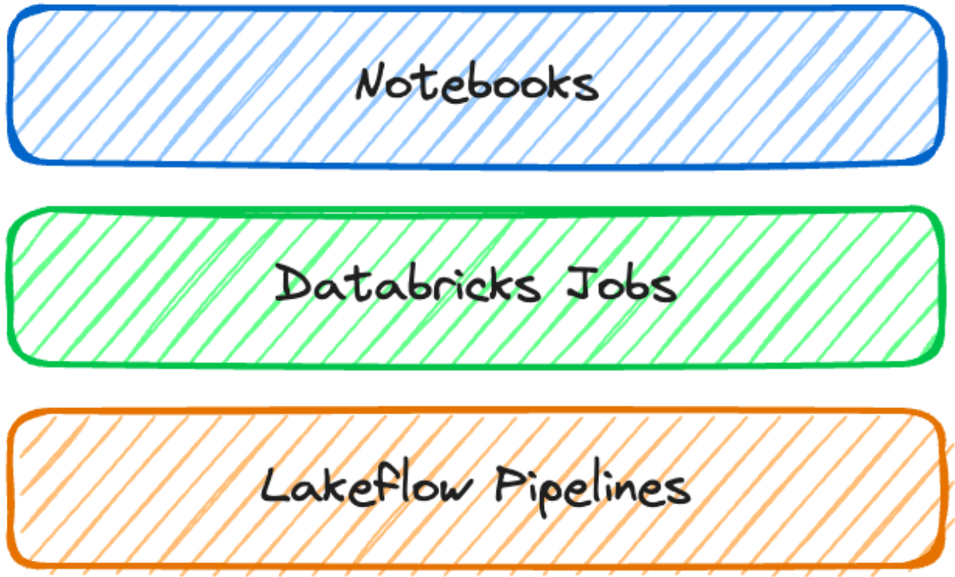
0 0 0

Filter by table | Type | Status

Name	Type	Duration	Output r...	Expectations	Dropped	Warnings	Failed	Incrementalization
category_revenue_gold	Materialized view	5s	6	Not defined	-	-	-	Full recompute
transactions_bronze	Materialized view	9s	100K	Not defined	-	-	-	Full recompute
transactions_silver	Materialized view	7s	33K	Not defined	-	-	-	Full recompute

- Bronze (100K rows) → Silver (33K rows) → Gold (6 rows)

Notebooks, Jobs, or Lakeflow?



- **Notebooks** → exploring, prototyping
- **Databricks Jobs** → multi-step, scheduled pipelines
- **Lakeflow** → fully managed, declarative

Let's practice!

DATA TRANSFORMATION WITH SPARK SQL IN DATABRICKS

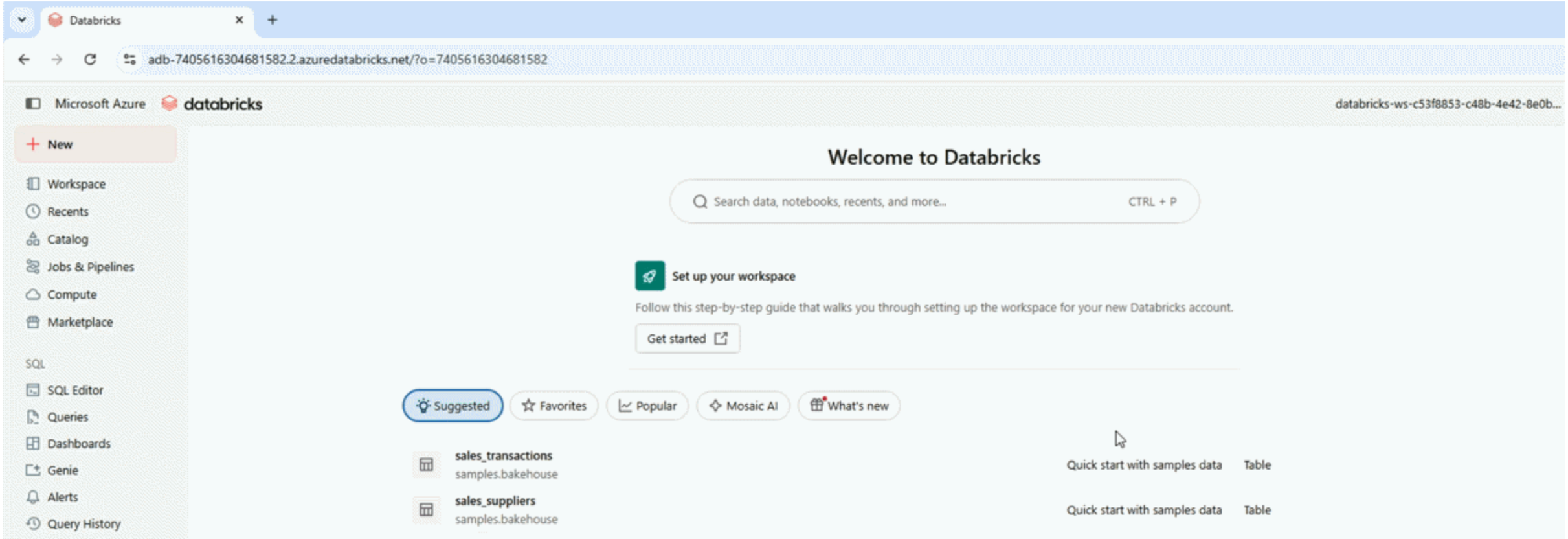
Wrapping up

DATA TRANSFORMATION WITH SPARK SQL IN DATABRICKS



Disha Mukherjee
Lead Data Engineer

Chapter 1



- Loading and inspecting DataFrames
- Shaping data with PySpark and Spark SQL

Chapter 2

```
total_rows = df_enriched.count()
distinct_rows = df_enriched.distinct().count()

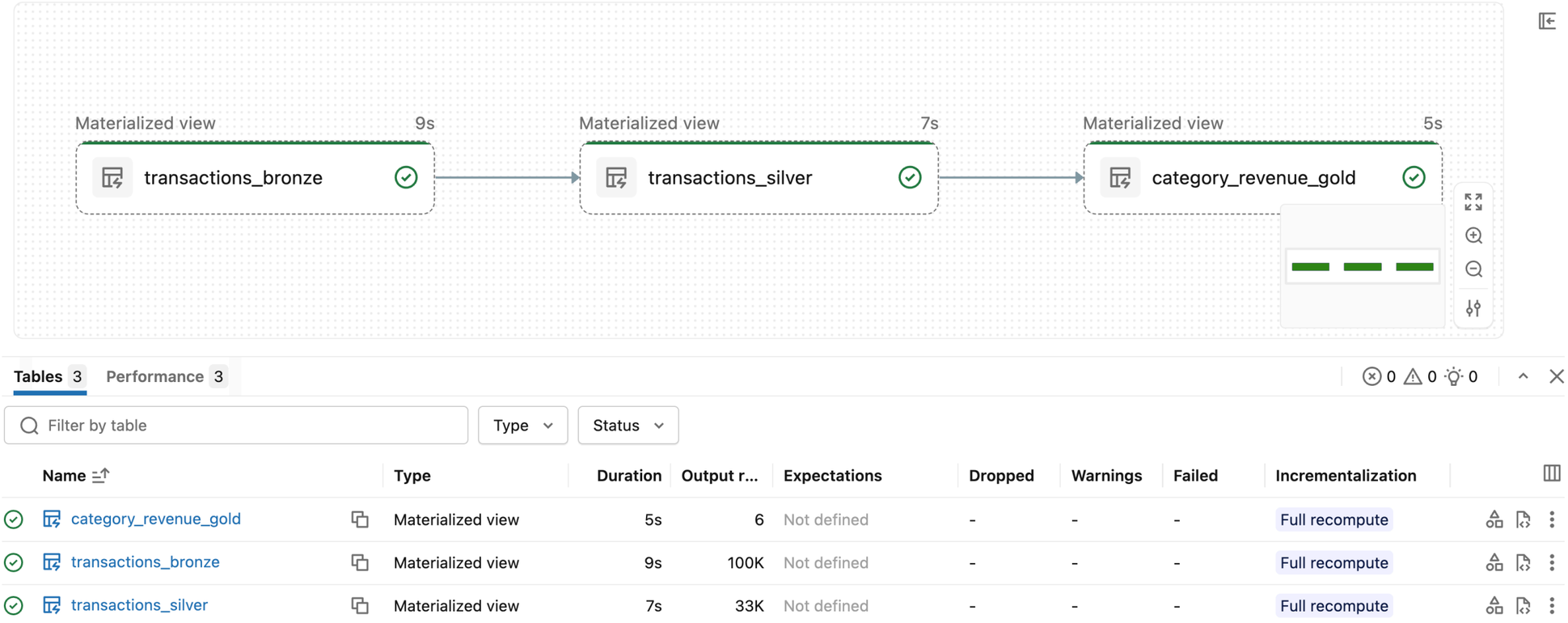
null_rate = (
    df_enriched.filter(F.col("Customer_ID").isNull())
    .count() / total_rows * 100
)

print(f"Total rows:   {total_rows}")
print(f"Duplicates:   {total_rows - distinct_rows}")
print(f"Null rate:    {null_rate:.2f}%")
```

```
Total rows:   33,223
Duplicates:   0
Null rate:    0.00%
```

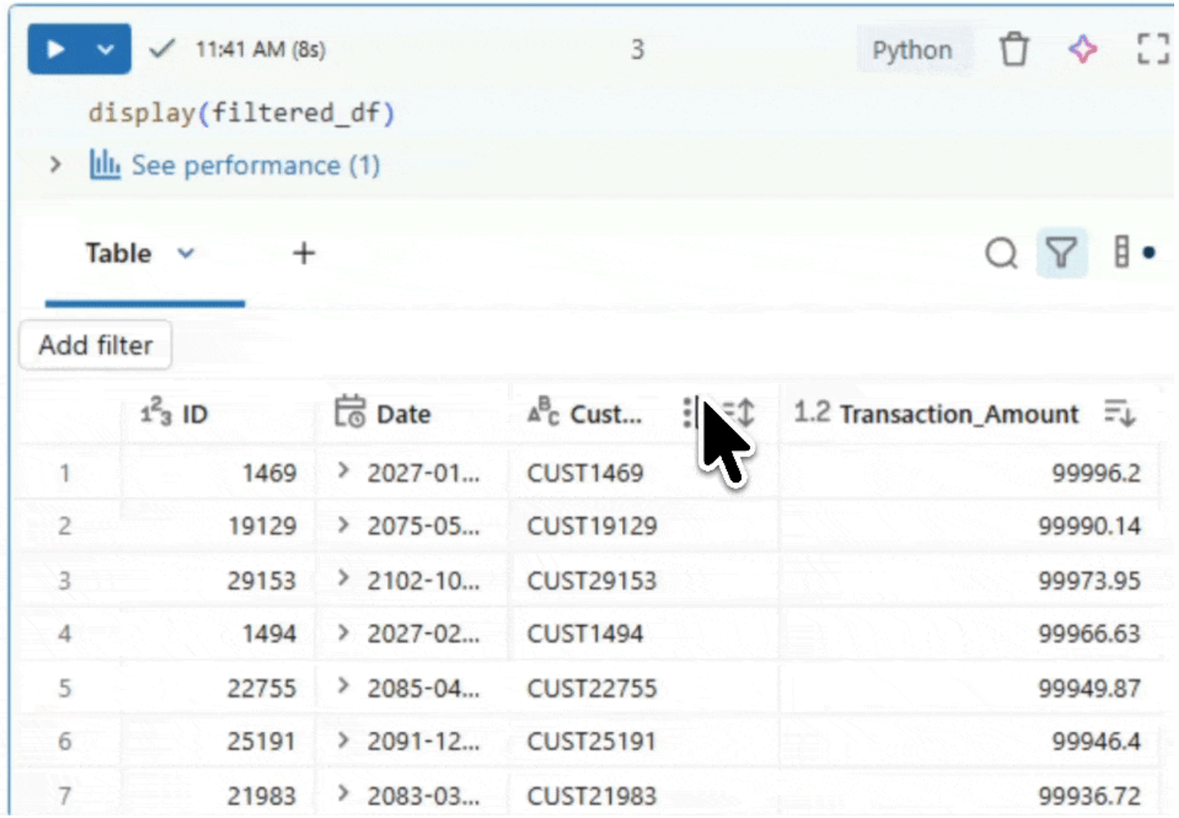
- Cleaning nulls, duplicates, and invalid records
- Aggregating and joining with broadcast optimization

Chapter 3



- Window functions and streaming queries
- Production pipelines with Databricks Jobs and Lakeflow

Practice in a live Databricks workspace



- Transactional and online retail datasets
- Interactive practice in a live Databricks workspace

Congratulations!

DATA TRANSFORMATION WITH SPARK SQL IN DATABRICKS